LIGHTWEIGHT RAY-TRACER

A PROJECT REPORT

submitted by

SATHISH K

(ROLL NO. 112401032)

for the

COURSE

CS4110 - BTP PROJECT

INDIAN INSTITUTE
OF TECHNOLOGY
PALAKKAD

Department of Computer Science and Engineering

CONTENTS

LIST OF FIGURES

LIST OF TABLES

Part I

OVERVIEW

1

INTRODUCTION

This project report describes the implementation of a lightweight and simple 3D renderer written in C++

1. The development of this framework serves as an exploration into the fundamental mechanics of light transport and the software engineering challenges of high performance computer graphics

2. The current framework by the end of the mid-semester is a full-fledged CPU ray tracer.

The main purpose of this project is to understand and develop a minimal and lightweight ray tracing framework while keeping a balance between quality, extensibility, and rendering speed. Here is the link to the gallery to see the final results. (see Section 4.5)

This project represents a minimal light-weight ray-tracer. Our initial implementation focuses on the use of CPU rendering. Our final objective is to make it real-time and interactive using a GPU by massively parallelizing the process.

1.1 WHAT IS RAY TRACING?

Ray tracing is a rendering technique that creates photo-realistic 3D computer graphics by simulating the behavior of light.

In the real world, light travels from sources (sun), bounces off objects and enters our eyes. this is how in reality we do see. If a computer tries to simulate this behavior, it is computationally intensive. Hence we approach this behavior backward, that is, we cast rays from our eyes (virtual camera) into the environment (scene) which then retrieves the color and bounces off accumulating color that our eye perceives based on the material of the surrounding.

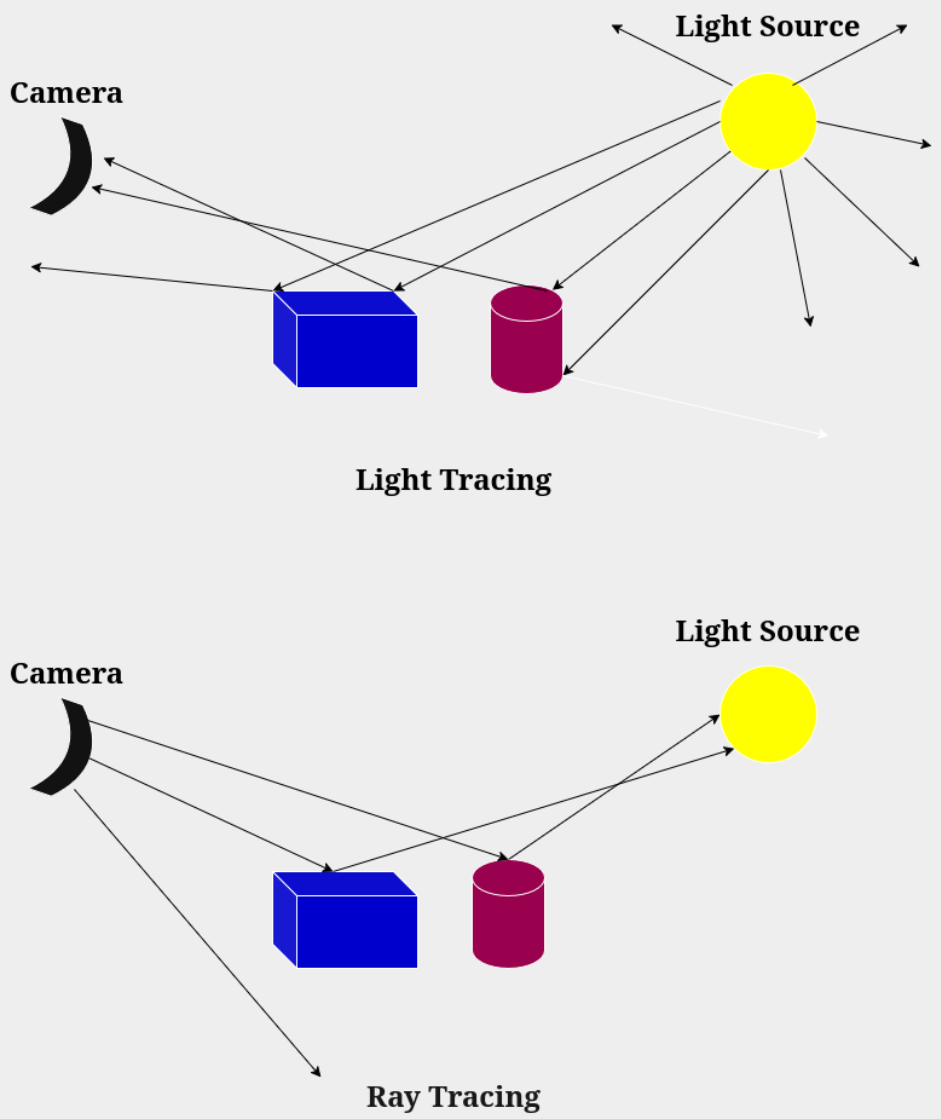

Figure 1.1: LightTracing vs RayTracing

1.2 RAY TRACING IN COMPUTER GRAPHICS

Ray tracing is widely used in many real-world applications, as it can simulate realistic lighting, reflection, refraction, shadows, and much more. Below are the popular use cases of a ray-tracer:

- `Film and Animation Industry`: Used to create visual effects and animated features where frames take hours to render for maximum realism.
- `Video Games`: Modern game titles use accelerated ray tracing for real-time reflections.
- `Architecture`: Allows architects to visualize how the interior of a building will look at different times of day.
- `Product Design`: Used in marketing imagery before even manufacturing as materials such as glass or chrome look stunning.
- `Virtual Reality`: Essential for presence and immersion, as accurate representation and lighting tricks the brain into accepting virtual environment as'real'.

1.3 OBJECTIVES OF THE PROJECT

The primary goal of this project is to architect a clean functional renderer which allows room for extensibility for people using this framework to adapt to their specific needs.

Currently, the project operates as a CPU bound renderer. However, the next phase of development aims to transcend these computational limits by rendering in real-time through the integration of CUDA.

1.4 SCOPE OF THE PROJECT

The engine is limited to the support of primitives such as spheres and quads as of now. But someone with more time and patience can go ahead and implement importing models as the foundations are laid out.

At the mid semester stage, the project exists as a fully functional CPU based renderer. It provides a complete pipeline for generating photorealistic static images.

As the core logic of the renderer is stabilized, the focus of the development process now moves towards achieving high performance through integration with GPU using the CUDA platform.

1.4.1 *Project Status*

Here are the implemented components as part of the project status till mid-semester:

- Implementation of BVH (Bounding Volume Hierarchy) and AABB (Axis Aligned Bounding Boxes) for optimized ray-scene intersection.
- Material system featuring Lambertian (diffuse), Metal (specular), dielectric and emmisive.
- Support for procedural checker patterns, solid colors and image based textures using uv co-ordinate mapping.
- Centralized scene context management for efficient memory handling of objects, Textures, and images.
- A virtual camera featuring stratified sampling, de-focus blur (depth of field) and adjustable vertical FOV.

1.5 ORGANIZATION OF THE REPORT

The report is structured to provide a concise overview of the project's architecture and it's deviations from the traditional implementations. The chapters are organized as follows:

- Mathematical Foundation: Review on the intersection logic for spheres and quads
- System Architecture: An overview of design differences and an analysis of the Ray-tracing Context approach that differs from the standard heap allocated pattern found in most introductory references.
- Spatial acceleration: A description of the BVH construction process, detailing how the engine handles AABB to maintain O(nlogn) performance.
- Materials: An overview of the scattering models used for metals, dielectrics, diffuse , and electromagnetic materials.
- Future Work and Conclusion: A summary of the current state and a roadmap for porting the kernel for hardware acceleration.

Part II

DESIGN AND IMPLEMENTATION

2

DESIGN AND IMPLEMENTATION

2.1 RENDERING WORKFLOW

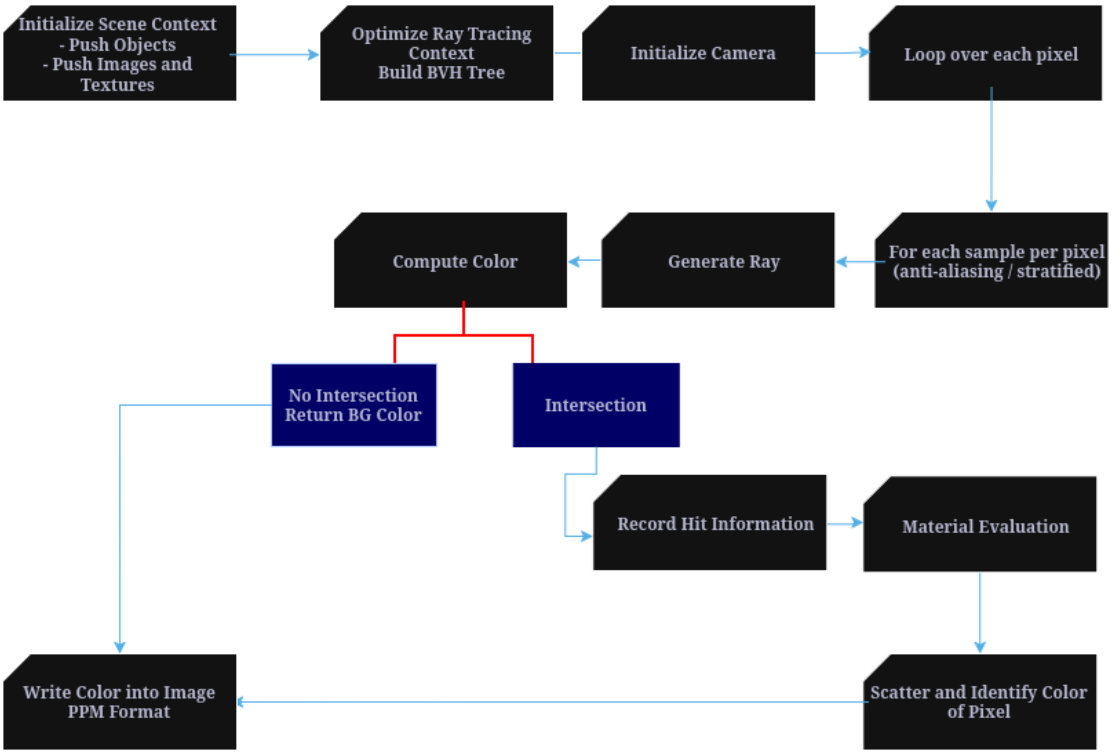

Figure 2.1: Rendering Workflow

The renderer outputs images in PPM (Portable Pixmap) format, following the Ray tracing. PPM is an image format where each pixel's color is determined by its respective RGB values written to the file. During rendering, the program computes the color of each individual pixels by casting rays and intersecting with objects. This approach avoids external libraries and makes it easier to understand and debug the core of the engine.

2.2 SCENE CONTEXT MANAGEMENT

The renderer uses a centralized scene context management to manage data related to the current rendering process. This is implemented through the ray tracing context

struct, which acts as a global container for geometry, acceleration structures, textures, and images used in the scene. By grouping all scene resources into a single container, the renderer maintains a design that improves organization and simplifies memory management.

The RaytracingContext stores a collection of hittable objects, images, textures, acceleration structures (BVH Nodes). The elements corresponding to these subcontainers are stored via calls to context functions such as PushHittable, PushTexture, and PushImage.

This approach is typically called a registry based approach as all resources are stored inside a register and are accesed via register indices.

This approach differs from introductory ray-tracers such as Ray tracing in one weekend, where scene elements are managed using std::sharedptr. Instead of relying on dynamic pointer ownership, the registry stores objects in contiguous memory and is accessed by indices.

This reduces pointer indirection, avoids reference counting overhead, and allows efficient use of resources such as textures and images across multiple objects.

Furthermore, this registry based layout was designed with GPU acceleration extensibilty in mind. Since GPU programming prefers contiguous buffers and index access to pointer indirection. Using this way, it is easier to extend this implementation for parallel rendering implementations using GPU.

```
// ********** RAYTRACING-CONTEXT **********
struct RaytracingContext {
  Hittable *hittables;

  size_t hittableSize;
  size_t hittableCapacity;

  // BVH
  BVHNode *bvhNodes;
  size_t bvhNodeSize;
  size_t bvhNodeCapacity;

  int bvhRootIndex;

  // Textures
  Texture *textures;
  size_t textureSize;
  size_t textureCapacity;

  // Images
  Image *images;
  size_t imageSize;
  size_t imageCapacity;

  // bounding box for the entire context scene
  AABB bBox;
};
```

Figure 2.2: RaytracingContext Struct

2.3 TAGGED-UNION APPROACH

The renderer makes use of the tagged union approach to inheritance instead of virtual functions and vtables as follows in the Raytracing in OneWeekend.

For example, Each hittable stores:

- a type enum: (SPHERE, QUAD, NONE)
- a union contianing the actual geometry data (Sphere or quad)

```
// ********** HITTABLE ********** //
// Tag for Geometry type
enum GeometryType { SPHERE, QUAD, NONE };

class Hittable {
public:
  GeometryType type;
  AABB bBox;

  union MemberData {
    Sphere sphere;
    Quad quad;
    // default constructors get destroyed placeholder constructors and
    // destrcutors manually handled via class constructors and destrcutors
    MemberData() {}
    ~MemberData() {}

  } data;

  Hittable();
  Hittable(Sphere s);
  Hittable(Quad q);

  ~Hittable();

  bool Hit(const Ray &r, Interval t, HitRecord &rec);
};
```

Figure 2.3: Example of tagged-union approach

when testing a ray for intersection, the renderer checks the enum's type associated with the object hit and calls the correct intersection function.

2.3.1 *Why vTables are not preferred?*

- Pointer Indirection: Each virtual call requires following vTable pointer, adding overhead to the CPU for each ray hit.
- Memory: Each object carries a hidden vTable pointer
- Extensibility: GPUs cannot easily handle vTables.

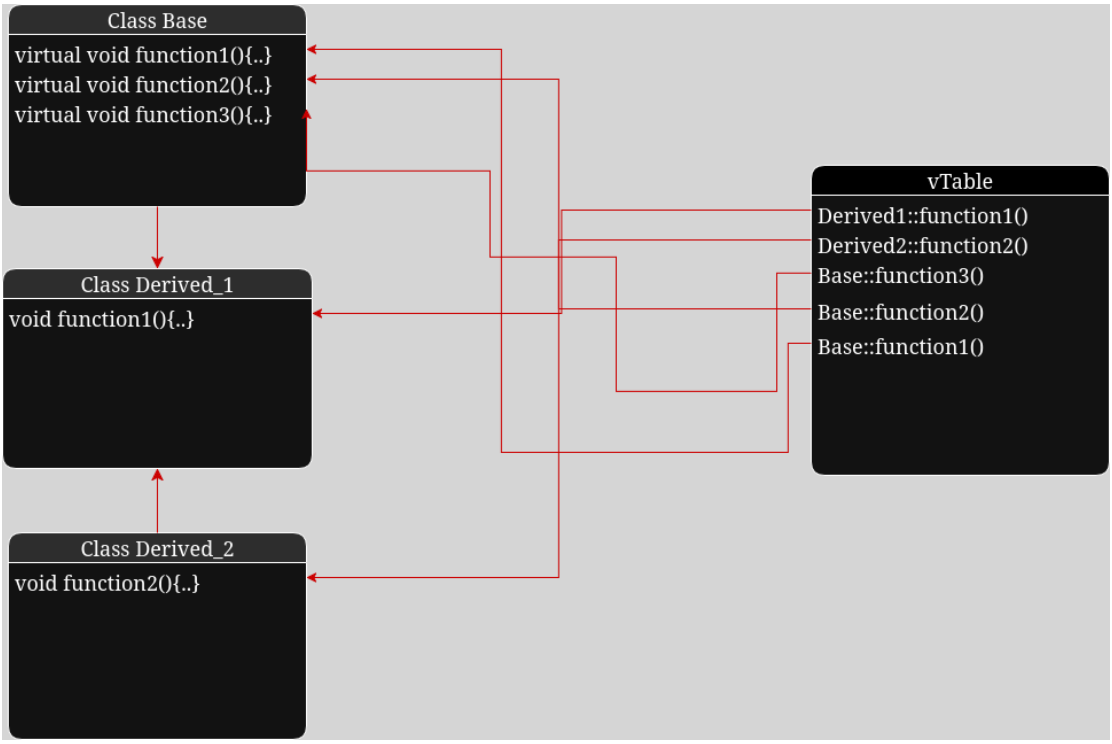

Figure 2.4: How vtable works

2.4 RAY-GEOMETRY INTERSECTION

This section covers how the ray geometry intersection works. The ray geometry intersection is the core of the renderer, each ray is tested to determine a hit on a scene object.

The intersection algorithm updates a HitRecord, storing hit information such as normal, material, and other relevant information. The Mathematical foundation is similar to Raytracing in One Weekend [1], but adapted to the tagged union geometry system.

2.4.1 *Intersection with Spheres*

The equation for a sphere of radius r that is centered at the origin is an important mathematical equation:

$$x^2 + y^2 + z^2 = r^2$$

This equation can be used along with basic ray math and determinant logic to determine which among the cases does the ray hit belong to as shown in figure 2.4

If it belongs to the case 1 root, then we retrieve the info at the solution point; else if it belongs to the case 2 roots, then we retrieve the closest solution information into

the hitrecord struct; otherwise, we detect no hit on the sphere object. Here is the math behind the intersection of the sphere with ray.

$$
\begin{aligned}
x^2 + y^2 + z^2 &= r^2 \\
(C_x - x)^2 + (C_y - y)^2 + (C_z - z)^2 &= r^2 \\
\mathbf{P} = (x, y, z), \quad \mathbf{C} &= (C_x, C_y, C_z) \\
(\mathbf{C} - \mathbf{P}) \cdot (\mathbf{C} - \mathbf{P}) &= r^2 \\
\mathbf{P}(t) &= \mathbf{Q} + t\mathbf{d} \\
(\mathbf{C} - \mathbf{P}(t)) \cdot (\mathbf{C} - \mathbf{P}(t)) &= r^2 \\
(\mathbf{C} - (\mathbf{Q} + t\mathbf{d})) \cdot (\mathbf{C} - (\mathbf{Q} + t\mathbf{d})) &= r^2 \\
(-t\mathbf{d} + (\mathbf{C} - \mathbf{Q})) \cdot (-t\mathbf{d} + (\mathbf{C} - \mathbf{Q})) &= r^2 \\
t^2(\mathbf{d} \cdot \mathbf{d}) - 2t(\mathbf{d} \cdot (\mathbf{C} - \mathbf{Q})) + (\mathbf{C} - \mathbf{Q}) \cdot (\mathbf{C} - \mathbf{Q}) &= r^2 \\
t^2(\mathbf{d} \cdot \mathbf{d}) - 2t(\mathbf{d} \cdot (\mathbf{C} - \mathbf{Q})) + (\mathbf{C} - \mathbf{Q}) \cdot (\mathbf{C} - \mathbf{Q}) - r^2 &= 0 \\
at^2 + bt + c &= 0 \\
a &= \mathbf{d} \cdot \mathbf{d} \\
b &= -2\,\mathbf{d} \cdot (\mathbf{C} - \mathbf{Q}) \\
c &= (\mathbf{C} - \mathbf{Q}) \cdot (\mathbf{C} - \mathbf{Q}) - r^2 \\
t &= \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \\
\Delta &= b^2 - 4ac
\end{aligned}
$$

2.4.2 *Intersection with Quads*

A quad (a quadrilateral) is defined by a corner point Q and two edge vectors u and v, forming a planar surface. Using the plane equation along with the parametric equation of a ray, the intersection point can be computed. By projecting the intersection point onto the local u and v axes of the quad, it is possible to determine whether the point lies within the boundaries of the quad, as illustrated in Figure 2.6.

The Materials and texture systems were done before the implementation of Quads, Hence the render example includes materials. The materials system is explored in 2.5 Material Systems.

The cross hatched purple lines represent the plane in which the desired quad lives, and the zigzag red pattern represents the actual quadrilateral. We can use the plane equation to determine the hit in the plane , then we can use the u and v vectors to decide whether the intersection point in the plane lies inside the quadrilateral or not.

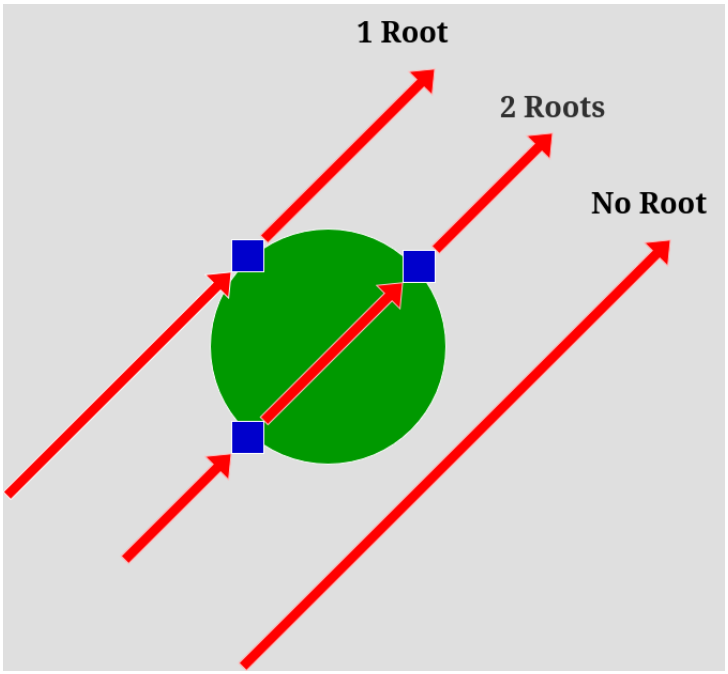

Figure 2.5: Sphere Intersection

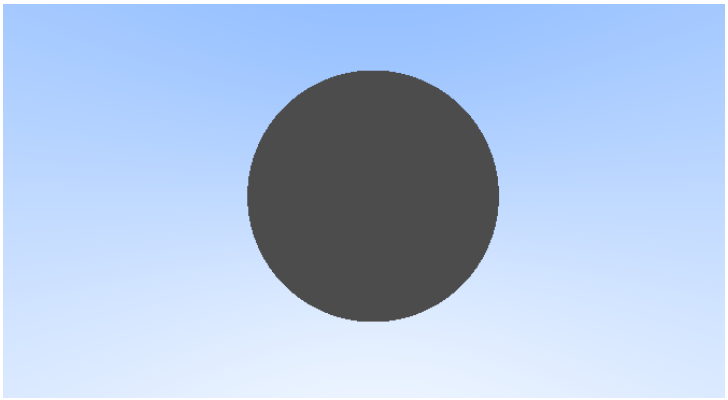

Figure 2.6: Example of Sphere Rendering

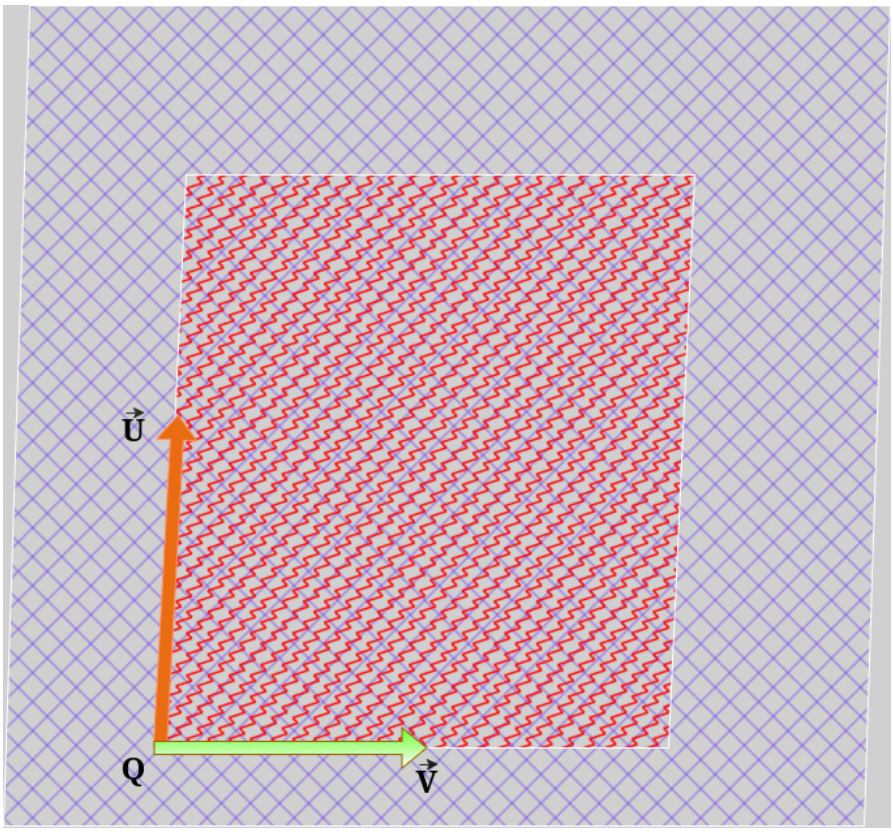

Figure 2.7: Quad Intersection

Figure 2.8: Example of Quad Rendering

Here is the math behind the quad intersection with the ray.

$$
\begin{aligned}
\mathbf{P}(t) &= \mathbf{O} + t\mathbf{d} \\
\mathbf{Q} &: \text{ quad origin} \\
\mathbf{u}, \mathbf{v} &: \text{ quad edge vectors} \\
\mathbf{n} &= \mathbf{u} \times \mathbf{v} \\
D &= \mathbf{n} \cdot \mathbf{Q} \\
\mathbf{n} \cdot \mathbf{P}(t) &= D \\
\mathbf{n} \cdot (\mathbf{O} + t\mathbf{d}) &= D \\
\mathbf{n} \cdot \mathbf{O} + t(\mathbf{n} \cdot \mathbf{d}) &= D \\
t &= \frac{D - \mathbf{n} \cdot \mathbf{O}}{\mathbf{n} \cdot \mathbf{d}} \\
\mathbf{P} &= \mathbf{O} + t\mathbf{d} \\
\mathbf{w} &= \frac{\mathbf{n}}{\mathbf{n} \cdot \mathbf{n}} \\
\alpha &= \mathbf{w} \cdot ((\mathbf{P} - \mathbf{Q}) \times \mathbf{v}) \\
\beta &= \mathbf{w} \cdot (\mathbf{u} \times (\mathbf{P} - \mathbf{Q})) \\
0 \leqslant \alpha &\leqslant 1 \\
0 \leqslant \beta &\leqslant 1
\end{aligned}
$$

Note: The same logic can also be applied for integration of triangles into the existing engine as later on for importing models, triangles will be a must-have if some one wishes to extend this implementation.

2.5 MATERIAL SYSTEM

The Material system defines how the ray interacts with the scene, determining reflection, refraction, and emmision.

Each Material stores type specific properties and implements a scatter function to compute ray behavior depending on the material of the object.

Unlike textures, hittables, or images these materials are not pushed onto the context structure; instead they are copied into the hittable objects itself.

It is just a design decision; one can move the materials also into the context and manage and use it in a registry based way.

```
class Material {
private:
  MaterialType type;

  union MemberData {
    Lambertian lambertian;
    Metal metal;
    Dielectric dielectric;
    Emmisive emmisive;

    MemberData() {};
    ~MemberData() {};
  } data;

  bool ScatterLambertian(const Ray &rIn, const HitRecord &rec,
                         Color &attenuation, Ray &scattered) const;
  bool ScatterMetal(const Ray &rIn, const HitRecord &rec, Color &attenuation,
                    Ray &scattered) const;
  bool ScatterDielectric(const Ray &rIn, const HitRecord &rec,
                         Color &attenuation, Ray &scattered) const;
```

Figure 2.9: Material Class

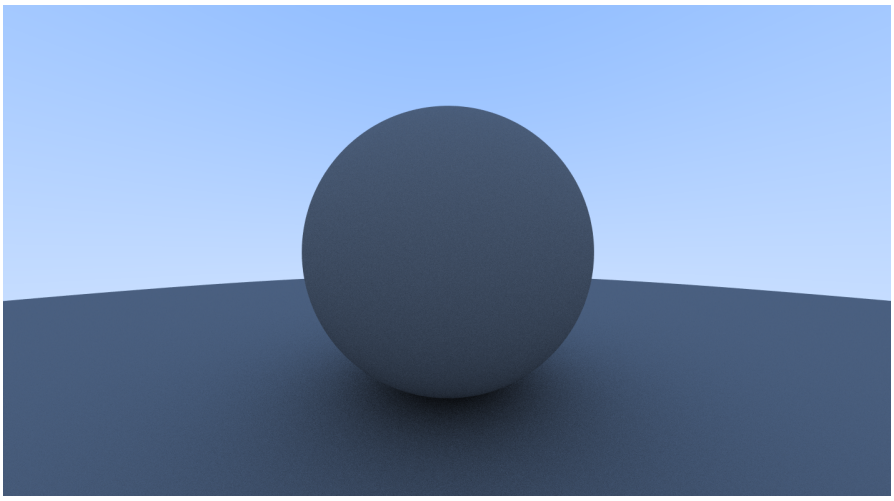

Figure 2.10: Basic Diffuse Sphere Scene

2.5.1 *Lambertian/Diffuse Material*

Lambertian materials scatter rays in all directions. The bounced off ray is generated by adding a random unit vector to the surface normal at the point of intersection. the color attenuation is obtained from the material's texture/albedo which is obtained via the id contained in the lambertian struct, allowing both solid colors and textured surfaces to be handled consistently.

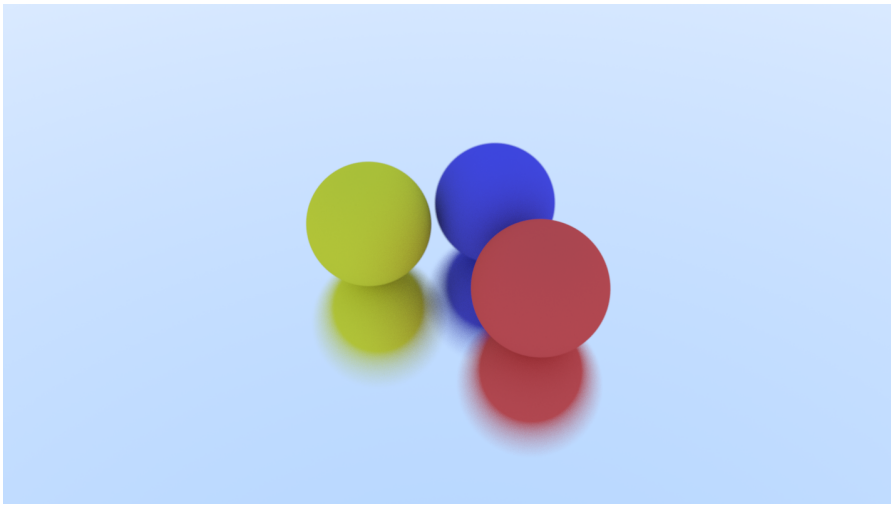

Figure 2.11: Glass Ground Scene With Lambertian Spheres

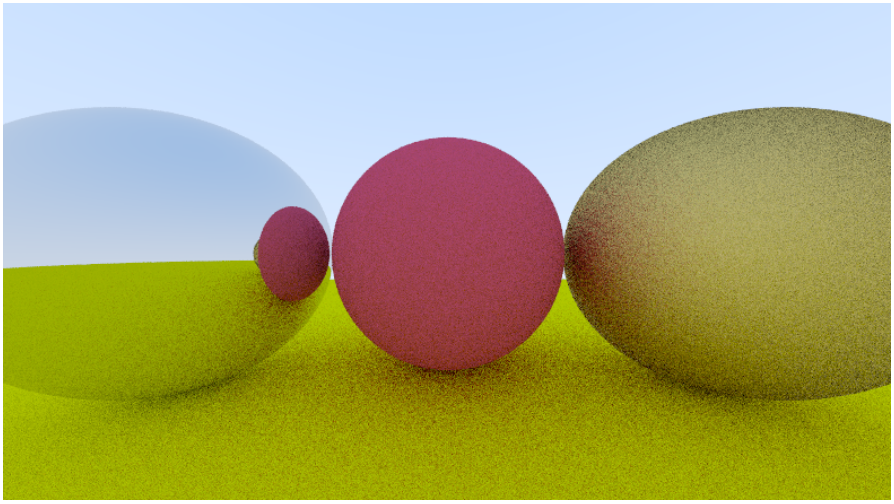

Figure 2.12: Metal Spheres with low and high roughness

2.5.2 *Metal Material*

Metal materials reflect rays primarily along the mirror direction. The bounced ray is computed by reflecting the incoming ray around the surface normal. To simulate imperfect reflections, the variable roughness is directly connected to the offset from the surface normal by adding a random unit vector, which simulates this behavior exactly. The color attenuation is taken from the metal's albedo, giving the surface its reflective color.

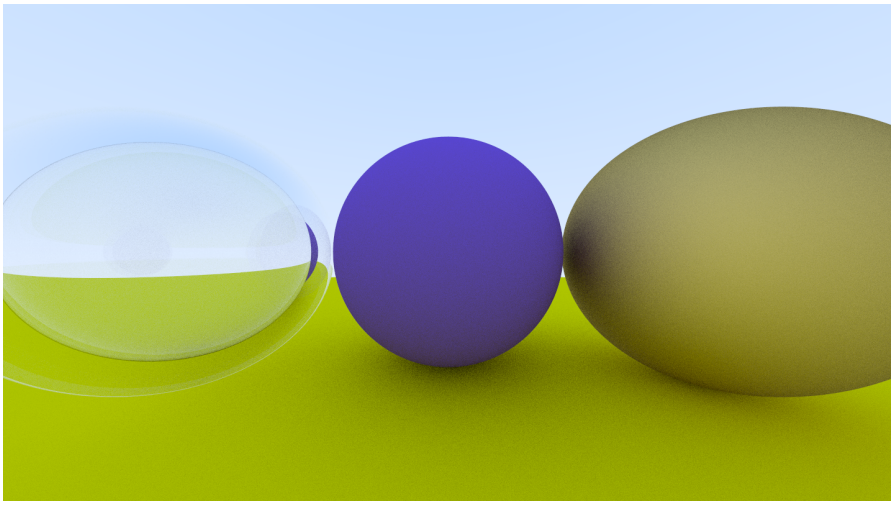

Figure 2.13: Bubble Sphere modelled via 2 concentric di-electric spheres

2.5.3 *Dielectric Material*

Dielectric materials model surfaces such as glass. Rays are either refracted or reflected on the basis of the refraction index and the incident angle. The Schlick approximation is used to probabilistically determine reflection versus refraction. The color attenuation is fixed to white because dielectrics do not absorb light but only bend or reflect it.

2.5.4 *Emmisive Material*

Emissive materials represent surfaces that emit light. The emitted color is obtained from the texture of the material, referenced via the ID stored in the emissive structure. This allows both solid-colored and textured light sources to be handled consistently in the scene.

2.6 TEXTURE SYSTEM

The Texture system defines how each point on the surface of an object has attributed colors. Each texture type implements a Value function that returns the color for the given the (u, v) co-ordinate. This method is usually called UV mapping. Textures are referenced via IDs in the material as textures are pushed on to the context in our implementation, allowing reuse for multiple objects.

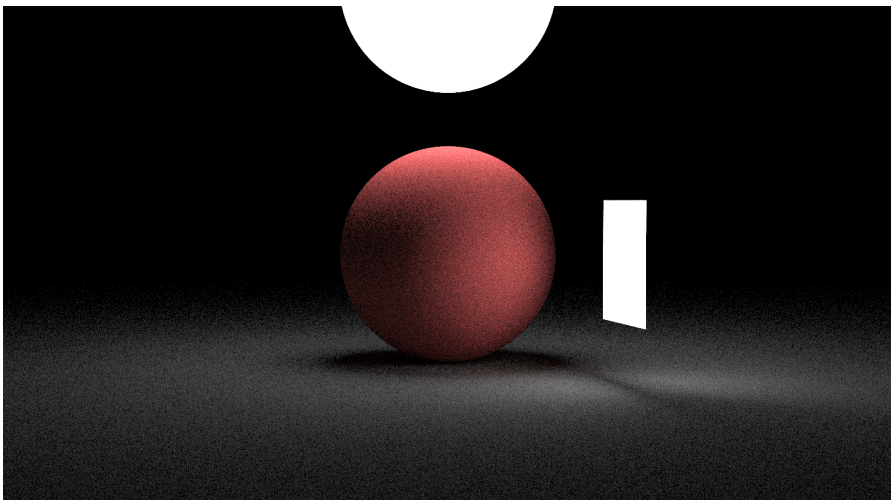

Figure 2.14: Emmisive material example

```
class Texture {
public:
  // Constructors
  Texture();
  Texture(const SolidTexture solidTexture);
  Texture(const CheckerTexture checkerTexture);
  Texture(const ImageTexture imageTexture);
  Texture(const Color color);

  Color Value(double u, double v, const Vec3 &p,
              const RaytracingContext *context) const;

private:
  TextureType type;
  union MemberData {
    SolidTexture solidTexture;
    CheckerTexture checkerTexture;
    ImageTexture imageTexture;

    MemberData() {}
    ~MemberData() {}
  } data;
};
```

Figure 2.15: Texture System

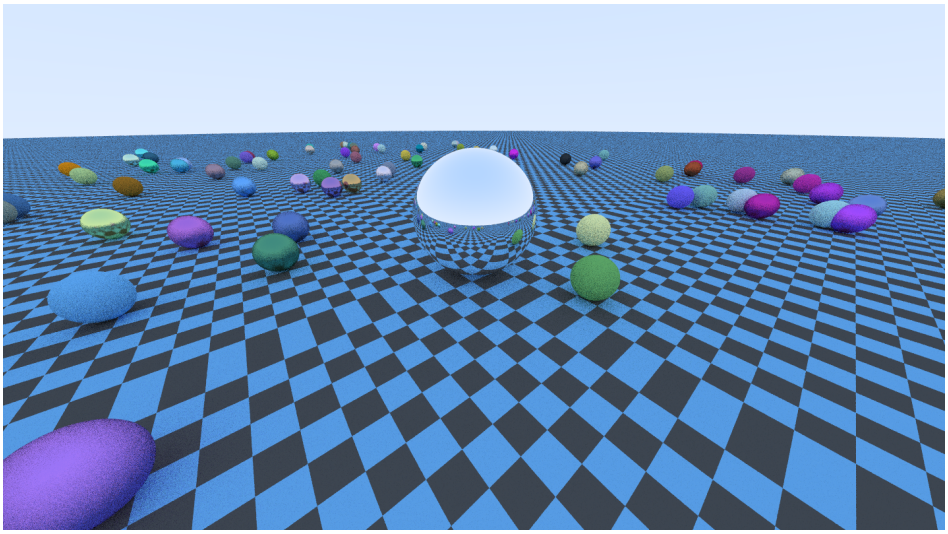

Figure 2.16: Checkered Ground - Random Scene Generation

2.6.1 *Solid Texture*

Solid textures represent a uniform color throughout the surface. The Value function returns the color stored in the texture, allowing materials to have a constant albedo.

2.6.2 *Checkered Texture*

The checked textures create a pattern that repeats across the surface. The Value function determines which color to return based on the scaled (u, v) coordinates and the IDs of the two alternating colors, producing a classic checkerboard effect.

2.6.3 *Image Texture*

Image textures map a 2D image onto the surface. The Value function samples the corresponding pixel from the loaded image using the (u, v) coordinates, allowing realistic and detailed textures to be applied to objects.

Note: Here the engine uses the STB image library for quickly setting things up. One could always go ahead and re-write his/her own implementation of image library such as STB.

Figure 2.17: Image loaded on a Sphere

Figure 2.18: Image loaded on a Quad

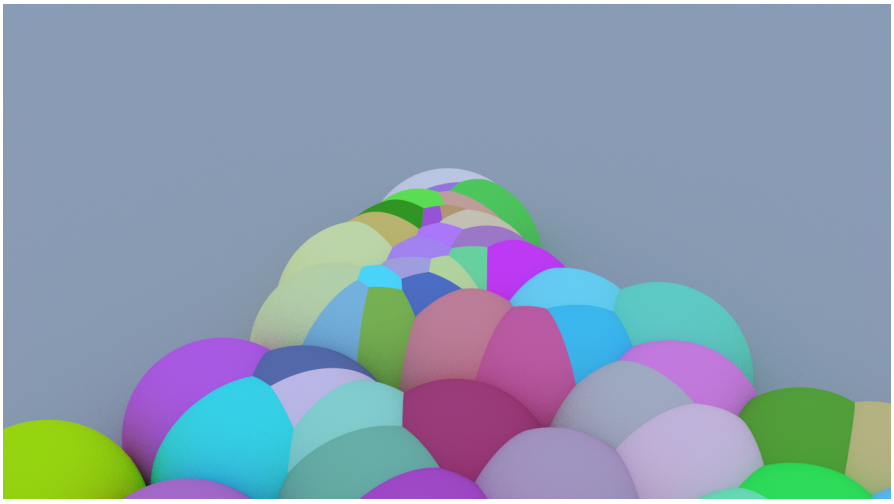

Figure 2.19: Pile of balls scene

2.7 ACCELERATION STRUCTURES - BVH NODES

A Bounding Volume Hierarchy (BVH) is an acceleration structure used to improve the performance of the ray tracer by optimizing the ray object intersection. Instead of testing a ray against all objects in the scene, objects are organized into a hierarchical tree of volumes, and hence the name Bounding Volume Hierarchy.

Each BVH Node contains an Axis Aligned boundary box that encloses a set of objects or child nodes. When a ray is cast, the renderer checks for intersection with the bounding box. If the ray hits the bounding box then the traversal continues to it's child nodes or the contained objects. If it does not intersect, then we know for sure that all the subtrees/objects inside that node can be skipped.

This hierarchy significantly simplifies the number of intersection tests, making rendering much faster for scenes with many objects. It is equivalently a time-space tradeoff. Memory is consumed to make the BVH in-order to reduce the time to reduce the number of intersections.

Note: There is no significant difference between the output with and without BVH Optimization.

Scene	With BVH (s)	Without BVH (s)	Speed Up %
Random Scene (Medium Sample)	21	70	70.00
Pile Of Balls (High Sample)	248	554	55.23
Pile Of Balls (Low Sample)	26	57	54.39

Table 2.1: Analysis Table

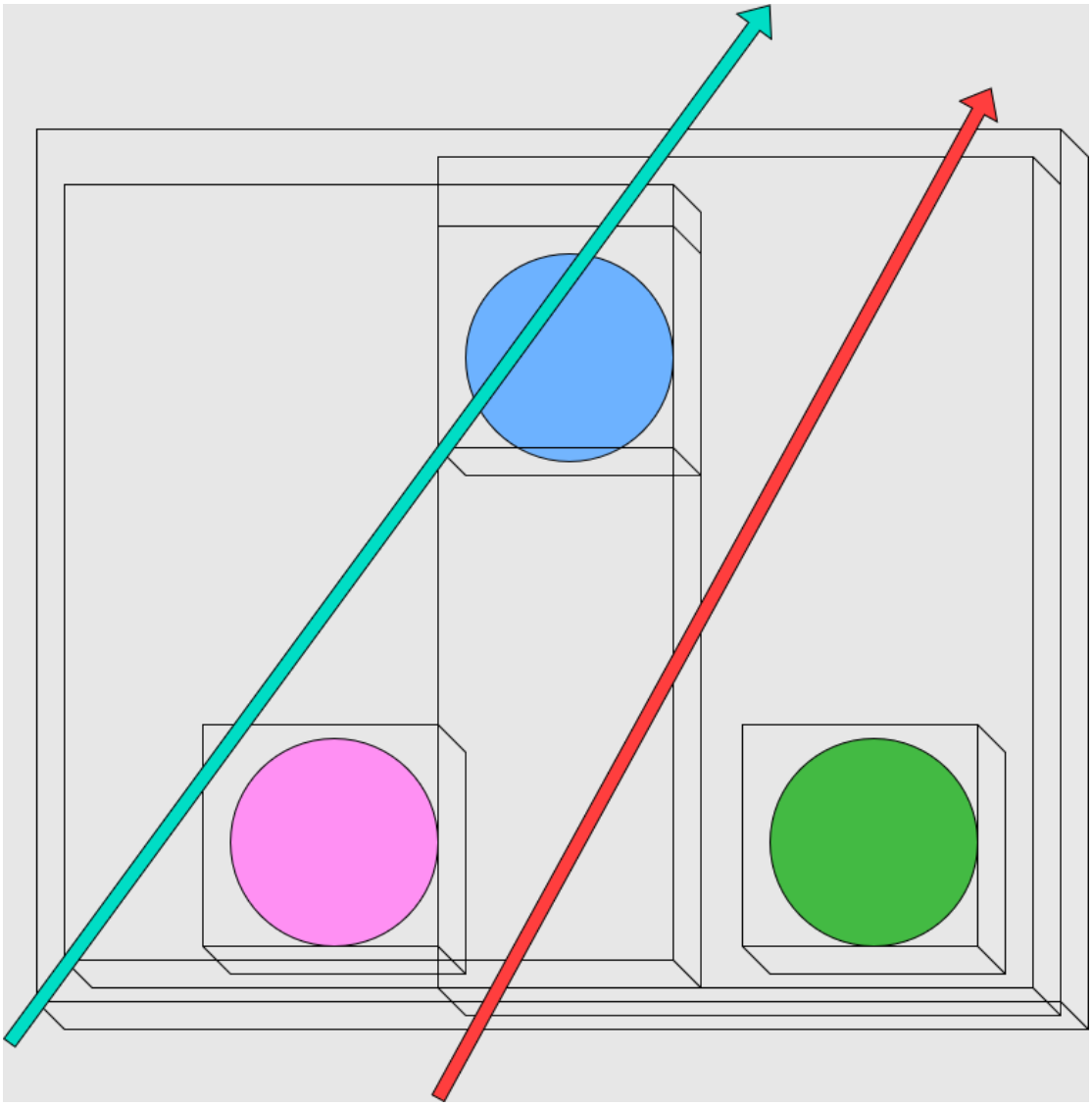

Figure 2.20: A Diagram Illustrating BVH Nodes with AABB

The pile of balls scene is literally just a pile of balls just to test how good the optimization works. it has hundred balls piling up on one over the other. From the analysis table, we could clearly see the potential profit in time for the memory we consume for the BVH Nodes.

As Figure 2.19 illustrates, the AABB (Axis Aligned Bounding Boxes) are stored inside the BVH Nodes in a tree format, which allows the engine to work in a time complexity of O(logn) for a ray-cast hit.

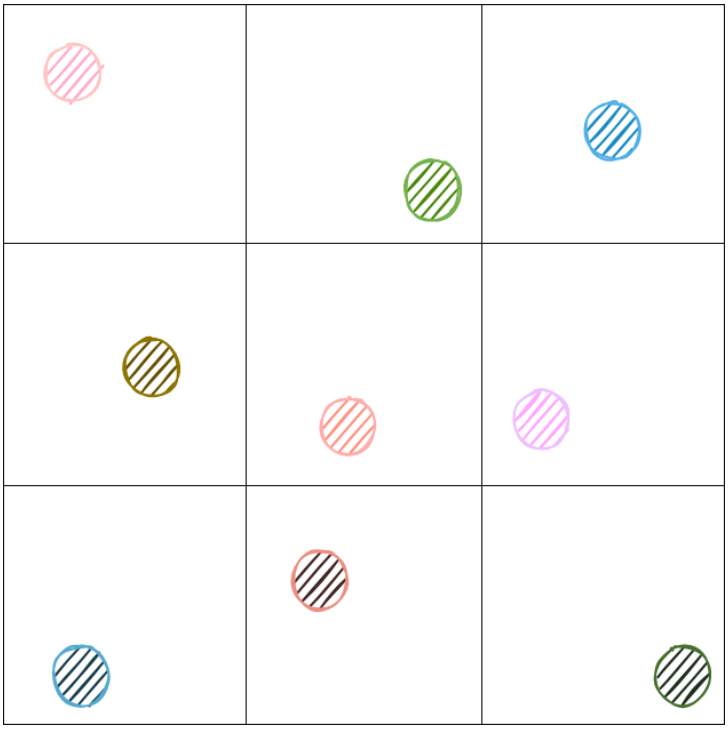

Figure 2.21: Stratified Sampling - Breaking down pixels into cells and sampling

2.8 CAMERA AND SAMPLING

This section deals with the features and techniques of the virtual camera modeled in the engine.

2.8.1 *Camera Workflow*

The virtual camera is defined using a look-from position and look-at target similar to the Raytracing in One Weekend. The image plane is positioned at a specified focus distance from the camera, and it's size is determined by vertical field of view. Rays are generated from the camera through points on this image plane to sample the scene.

2.8.2 *Stratified Pixel Sampling*

To reduce aliasing effects, multiple samples are taken per pixel using stratified sampling. Each pixel is subdivided into a grid of smaller regions, and a random sample is taken within each cell. This ensures a more uniform distribution of samples compared to purely random sampling.

the average of all the contributions produces the final pixel color.

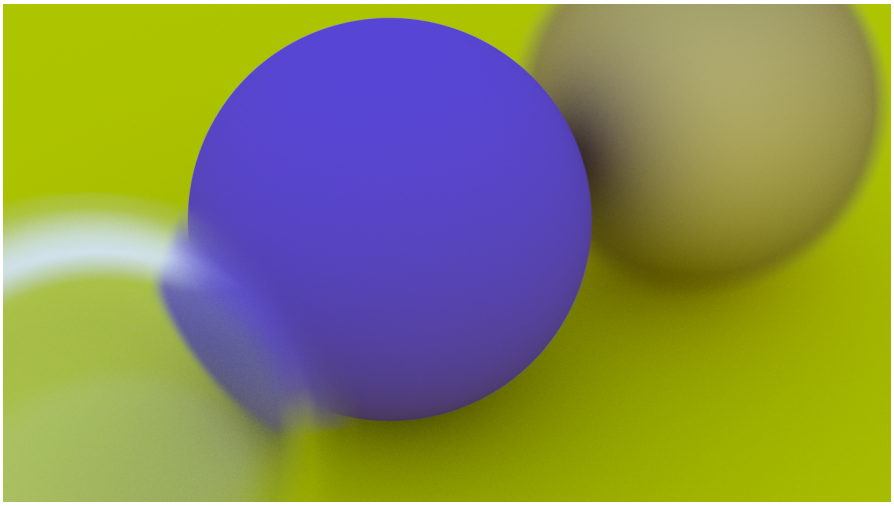

Figure 2.22: Focused Ball

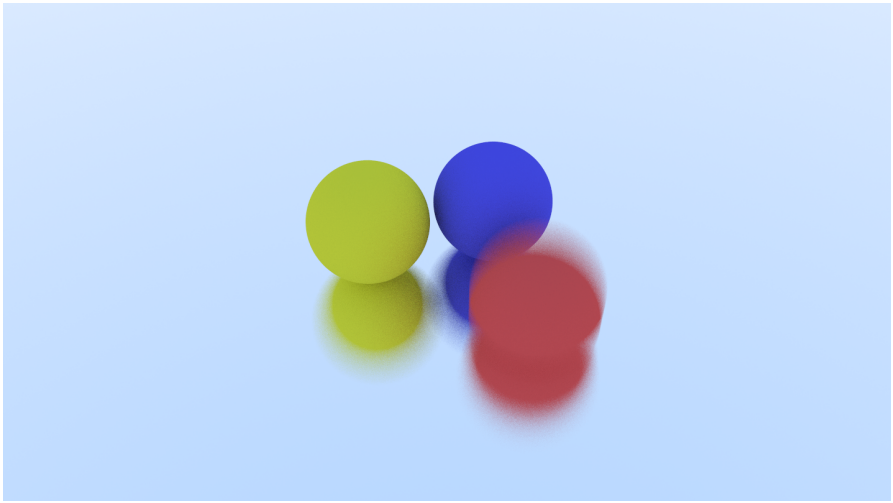

Figure 2.23: Moving Ball

2.8.3 *Depth Of Field and Motion Blur*

The camera also simulates depth of field using a defocus disk. Instead of all rays originating from a single point, the rays are sampled from a circular region that represents the camera aperture. The ray is then directed toward the focal plane, causing objects away from the focus distance to appear blurred.

To reproduce this effect, each ray is assigned a random time value within the shutter interval. Objects in the scene can change their position based on this time parameter, meaning that the ray may intersect the object at a slightly different location depending on when the ray was generated. By sampling multiple rays at different times and averaging their contributions, the renderer produces the appearance of a continuous motion.

Part III

PERFORMANCE OPTIMIZATION

3

CPU - PARALLELIZATION

The computational complexity of ray-tracing comes from the fact that each pixel has to be colored which takes multiple samples and each sample involves multiple ray-scene intersection tests. To speed up the renderer, OpenMP is utilized for CPU based machines (mainly machines which do not faciliate a GPU) to utilize the multi-core architecture

3.1 MOTIVATION FOR PARALLELISM

Ray tracing is a parallel program. Since the color of one pixel does not depend on the color of another, each pixel can be computed independently. By parallelizing with the workload, we can reduce the rendering time linearly with the CPU cores available on the machine.

3.2 IMPLEMENTATION OF OPENMP

By collapsing the height two loops for the CPU renderer, OpenMP treats the entire 2D Image grid as a single 1D array of work iterations. This parallelizes the work of the renderer across the multiple cores available in the architecture. **16 Threads** are active, which are physically bound by the **10 underlying cores** in the rendering device used for the analysis.

3.2.1 *Guided scheduling*

The implementation utilizes guided scheduling, To achieve optimal performance, several OpenMP scheduling strategies were experimented. Ray tracing is prone to imabalance of workload amongst pixels. For example, a thread calculating the sky color finishes faster by returning the color of the sky directly without bouncing off objects like other rays.

The following experimental data were gathered to determine the most efficient configuration.

```cpp
void Renderer::RenderCPU(const Camera &camera, RaytracingContext *context,
                         FrameBuffer &frameBuffer) {

  // Initialize context backend
  context->renderMode = RenderMode::CPU;
  context->kernelContext = nullptr;

#pragma omp parallel for collapse(2) schedule(guided)
  for (int i = 0; i < camera.imgH; i++) {
    for (int j = 0; j < camera.imgW; j++) {
      Color pixelColor(0, 0, 0);

      for (int sj = 0; sj < camera.sqrtSpp; sj++) {
        for (int si = 0; si < camera.sqrtSpp; si++) {
          Ray r = camera.GetRay(j, i, si, sj);
          pixelColor += camera.RayColor(r, camera.maxDepth, context);
        }
      }
      frameBuffer.WriteToBuffer(camera.pixelSamplesScale * pixelColor, i, j);
    }
  }
}
```

Figure 3.1: Material Class

Configuration	Rendering Time (s)	Improvement (%)
collapse(2) [static by default]	238	baseline
schedule(dynamic, 1)	219	8.0
schedule(guided)	216	9.2
schedule(guided) collapse(2)	213	10.5

Table 3.1: Analysis Table

3.2.2 *Reasoning*

Dynamic scheduling offers high flexibility, but it introduces significant scheduling overhead because threads must constantly request new work once the assigned work is finished by the thread. Static scheduling suffers from the idle problem stated as before. Guided scheduling was found to be the best algorithm among them.

3.3 THREAD SAFETY

The entire ray tracing context is read-only during the render phase, hence, this eliminates the need for mutex locks or atomic operations, allowing threads to work at 100% efficiency without synchronization issues.

4

GPU - PARALLELIZATION

Although OpenMP utilizes 8-16 cores on most modern architectures, a modern GPU can execute thousands of threads simultaneously. The GPU Implementation required a fundamental shift from object oriented approach to a data-parallel approach implemented through CUDA's programming model.

4.1 KERNEL WORKFLOW

The rendering process on GPU is managed by the RenderGPU function, which is used as the entry point to the GPU backend.

- **Context Upload:** GPU Context Manager takes the CPU Raytracing context and transfers the entire registry system onto GPU's memory.
- **Frame Buffer**: A GPU Frame Buffer is allocated directly on the device memory to store the results on the device memory rather than slow, per pixel transfers during the render.
- **Kernel Execution**: The host initializes the random number generator states and executes the Render Kernel.
- **Retrieval**: Once the GPU works on it's tasks, the host waits for device to finish it's work by cudaDeviceSynchronize() before copying the result from the device to host frame buffer.

4.2 TECHNICAL IMPLEMENTATION

The implementation relies on two primary CUDA kernels to handle the complexities of ray tracing.

4.2.1 *Parallel Random Number Generation*

Ray tracing requires high-quality random numbers for anti-aliasing and roughness reflections. Since standard C++ random generators are not thread-safe for parallel

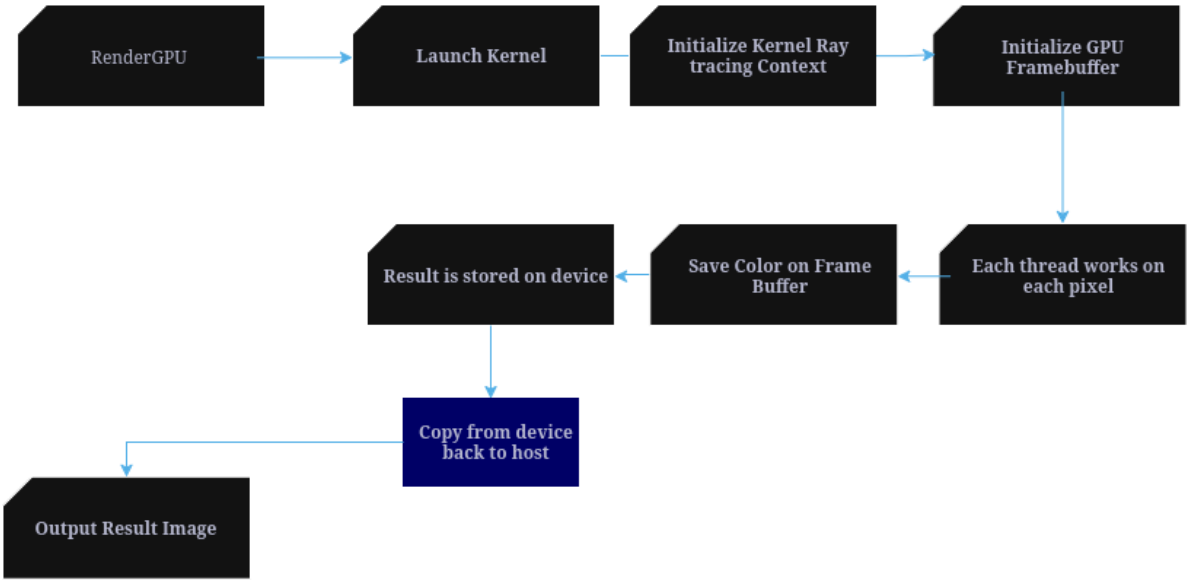

Figure 4.1: Kernel Workflow

execution, we utilized the curand library. The InitializeKernelRaytracingContext kernel assigns a unique curandState to every pixel. we ensure that every thread generates a unique, non-overlapping sequence of random numbers.

4.2.2 *The Render Kernel*

The main logic is in RenderKernel. Unlike the CPU version, which uses nested loops to iterate through pixels, the GPU kernel leverages the hardware's ability to run thousands of threads. Each thread identifies its target pixel using the built-in variables blockIdx and threadIdx. Threads outside the image boundaries are terminated via bounds checking to ensure memory safety and prevent illegal memory access.

4.3 THREAD GEOMETRY AND LAUNCH CONFIGURATION

We utilize a 2D grid block structure to maximize the occupancy of GPU ensuring that the threads are occupied.

- **Block Dimensions**: 16x16 block size (256 Threads).
- **Grid Calculation**: The grid dimensions are calculated dynamically to cover the entire image resolution. Rounding up the size of the grid ensures that every pixel is assigned to a thread.

Configuration for Cornell Box scene:

```cpp
__global__ void RenderKernel(const Camera camera,
                             const RaytracingContext *deviceContext,
                             GPUFrameBuffer deviceFb) {

  int imageWidth = camera.imgW;
  int imageHeight = camera.imgH;

  int workIndexX = (blockDim.x * blockIdx.x) + threadIdx.x;
  int workIndexY = (blockDim.y * blockIdx.y) + threadIdx.y;

  // Bounds Checking - Invalid Threads
  if (workIndexX >= imageWidth || workIndexY >= imageHeight)
    return;

  Color pixelColor(0, 0, 0);
  int workIndex = (workIndexY * imageWidth) + workIndexX;

  curandState *state = &(deviceContext->kernelContext->states[workIndex]);

  for (int sj = 0; sj < camera.sqrtSpp; sj++) {
    for (int si = 0; si < camera.sqrtSpp; si++) {
      Ray r = camera.GetRay(workIndexX, workIndexY, si, sj, state);
      pixelColor += camera.RayColor(r, camera.maxDepth, deviceContext, state);
    }
  }
  deviceFb[workIndex] = camera.pixelSamplesScale * pixelColor;
}
```

Figure 4.2: Render Kernel

- Blocks in X: $\lceil 1440/16 \rceil = 90$ blocks
- Blocks in Y: $\lceil 810/16 \rceil = 51$ blocks

Total Threads Launched:

The total number of threads launched is the total number of blocks multiplied by threads per block:

$$\text{Total Threads} = (90 \times 51) \times 256$$

$$\text{Total Threads} = 4,590 \text{ blocks} \times 256 \text{ threads/block}$$

$$\text{Total Threads} = \mathbf{1,175,040}$$

4.4 RESULTS

The transition to CUDA provided a massive performance jump compared to the CPU. By transferring the expensive per pixel work load to GPU, we transformed a few minute render into a few second process.

Cornel box scene recreated with different samples per pixel to analyze the improvement from CPU to GPU

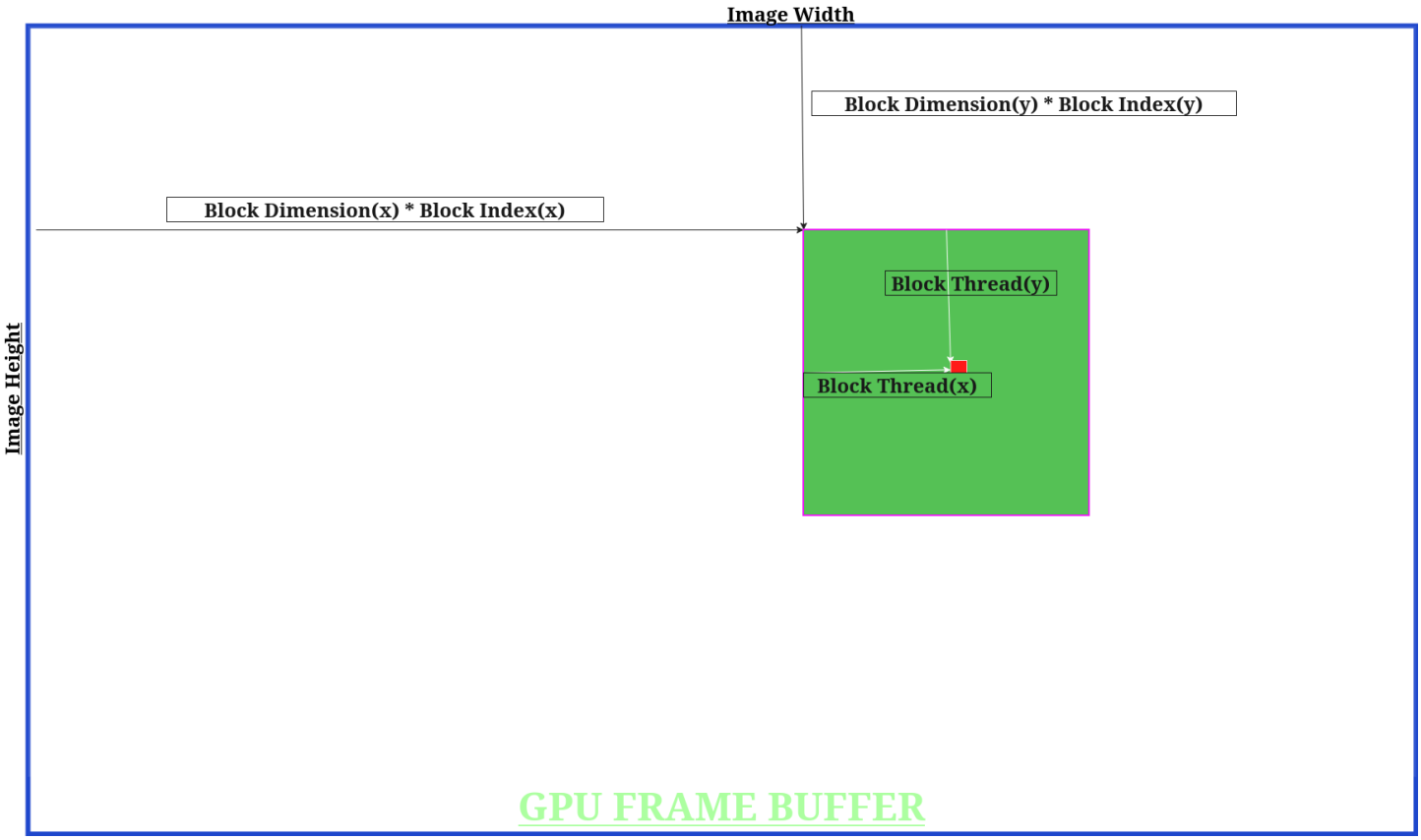

Figure 4.3: Illustration of Configuration

Samples	Rendering Time GPU (s)	Rendering Time CPU (s) (%)
100	10	40
1000	71	427
5000	357	2237

Table 4.1: Analysis Table

Part IV

GALLERY

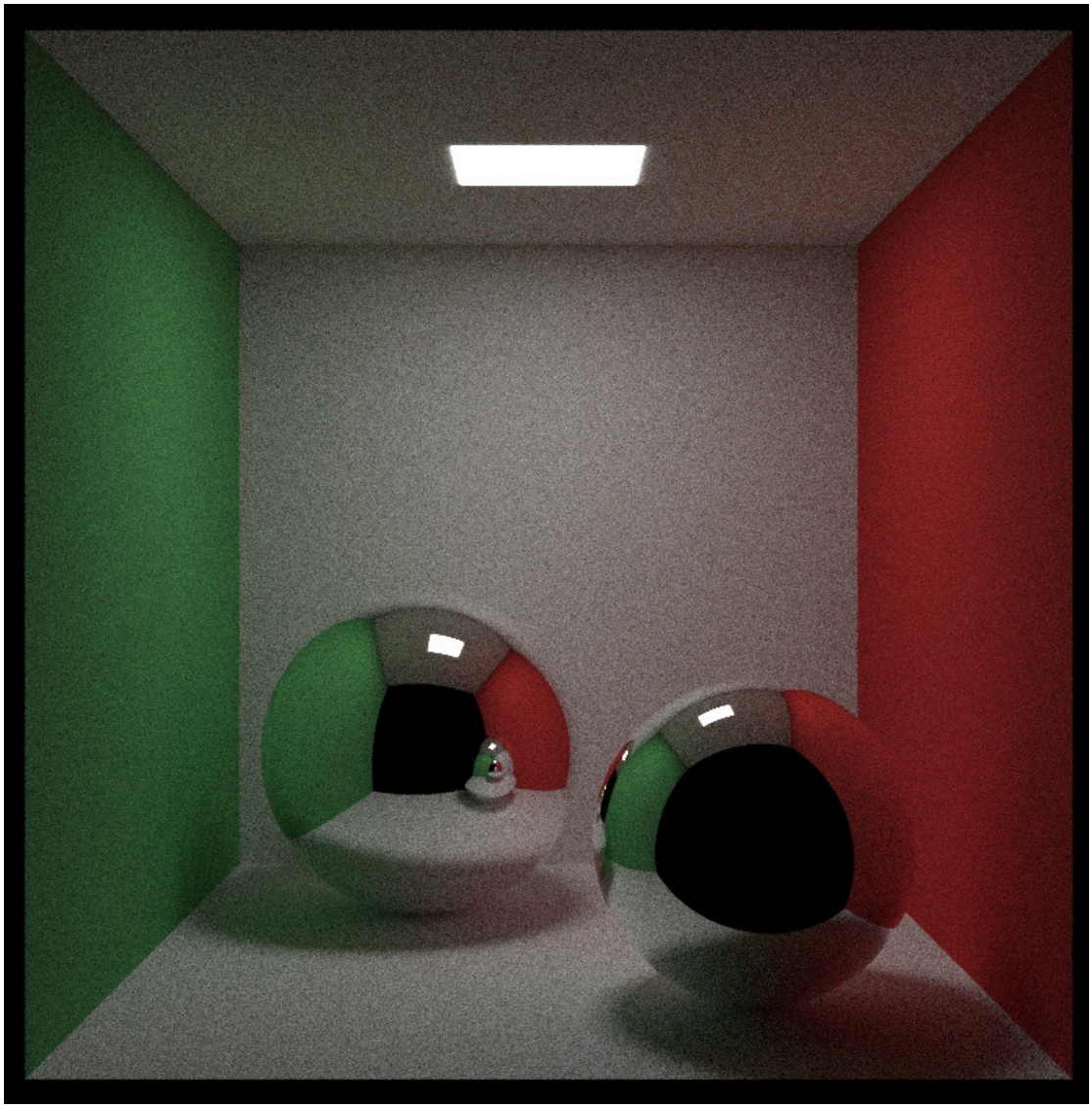

Figure 4.4: Cornell Box

4.5 GALLERY

This section showcases the renders

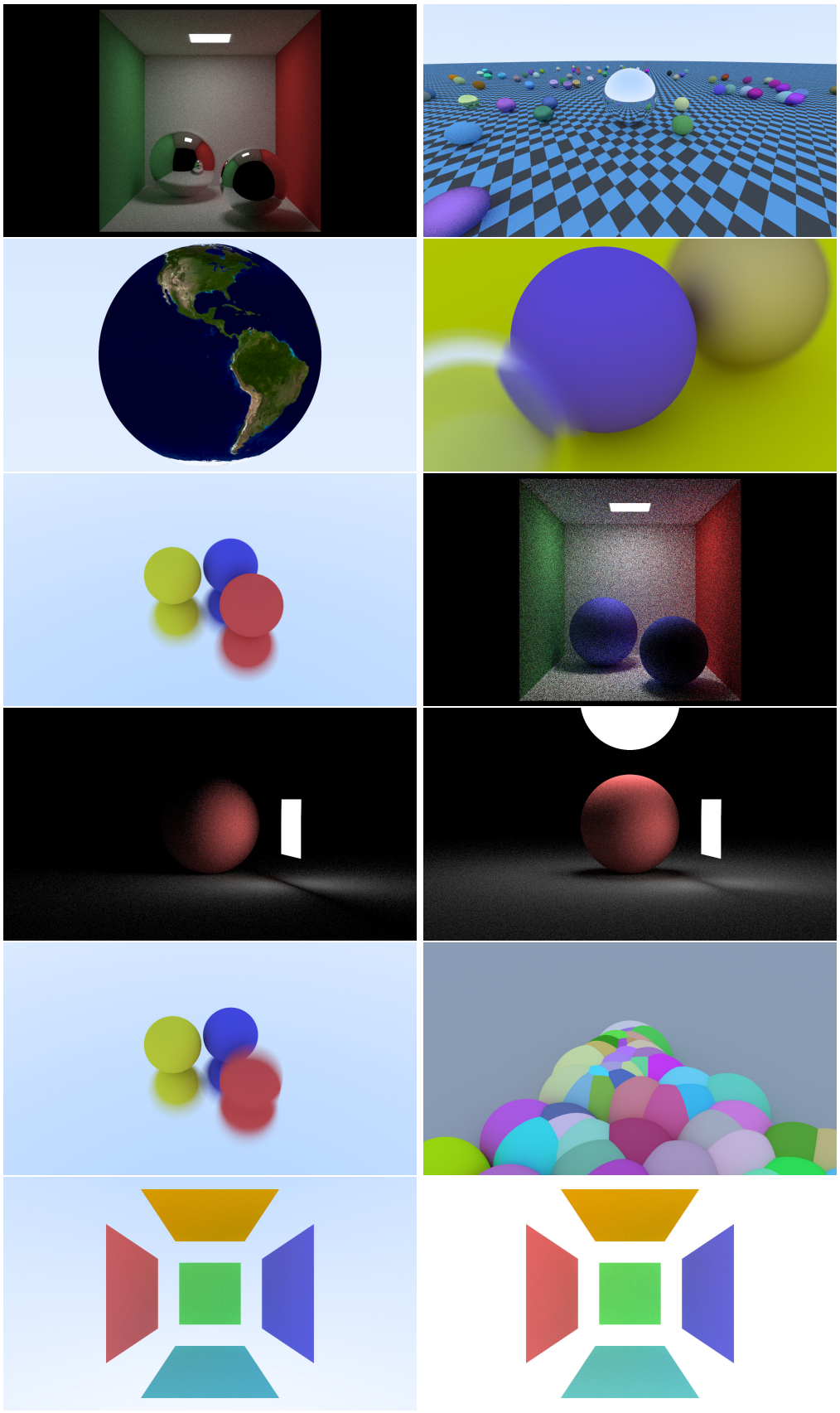

Figure 4.5: Gallery of rendered scenes demonstrating camera and sampling features.

BIBLIOGRAPHY

[1] Peter Shirley. *Ray Tracing in One Weekend*. Ray Tracing in One Weekend Series, 2016. URL: https://raytracing.github.io/books/RayTracingInOneWeekend.html.